you remember the biology)—only matched chromosomes will recombine. Using this approach, you would only recombine parameters from the same layer. This would prevent information from later layers corrupting the earlier layers. We encourage you to try to implement it using this "chromosomal" approach as a challenge once you're comfortable with the way we do it here. You'll need to iterate over each layer, recombine, and mutate them separately.

Next let's add a function to create a population of agents.

Listing 6.7. Spawning a population

```
def spawn_population(N=50,size=407):          1
    pop = []
    for i in range(N):
        vec = torch.randn(size) / 2.0         2
        fit = 0
        p = {'params':vec, 'fitness':fit}     3
        pop.append(p)
    return pop
```

- ***1*** N is the number of individuals in the population; size is the length of the parameter vectors.
- ***2*** Creates a randomly initialized parameter vector
- ***3*** Creates a dictionary to store the parameter vector and its associated fitness score

Each agent will be a simple Python dictionary that stores the parameter vector for that agent and the fitness score for that agent.

Next we implement the function that will recombine two parent agents to produce two new child agents.

Listing 6.8. Genetic recombination

```
def recombine(x1,x2):                         1
    x1 = x1['params']                         2
    x2 = x2['params']
    l = x1.shape[0]
    split_pt = np.random.randint(l)           3
    child1 = torch.zeros(l)
    child2 = torch.zeros(l)
    child1[0:split_pt] = x1[0:split_pt]       4
    child1[split_pt:] = x2[split_pt:]
    child2[0:split_pt] = x2[0:split_pt]
```